# EPADASH

# Task 2 — Core Backend Services, Authentication and User Management

Implementation guide · Version 0.1

Deliverable: Backend starter implementation package

Status: Starter code prepared; schema integration, provider wiring and runtime verification pending

Baseline: Node.js 20+, TypeScript, Express, PostgreSQL, Prisma

## 1. Scope covered

This package provides a modular Express/TypeScript foundation for:

• Customer registration and login.

• Short-lived JWT access tokens.

• Opaque refresh tokens stored as hashes, with rotation and revocation.

• Logout of the current refresh session.

• Password reset token generation and password replacement service logic.

• User profile retrieval and permitted profile updates.

• Role-based route protection and administrator account-status changes.

• Request validation, rate limiting, Helmet, CORS and centralized error responses.

## 2. Security choices in the starter

• Argon2id is used for password hashing.

• Refresh/reset tokens are random opaque values; only SHA-256 token hashes are stored.

• Access tokens are signed with a configurable secret and short default lifetime.

• Refresh token is transported in an HttpOnly cookie.

• Login returns a generic invalid-credentials response.

• Password recovery returns a generic acceptance response to reduce account enumeration.

• Request bodies are size-limited and validated with Zod.

• Administrative status changes are role-gated; audit-log integration remains a required production task.

## 3. API endpoints

| Method | Endpoint | Purpose |

| --- | --- | --- |

| POST | `/api/v1/auth/register` | Register a customer |

| POST | `/api/v1/auth/login` | Authenticate |

| POST | `/api/v1/auth/refresh` | Rotate refresh session and issue access token |

| POST | `/api/v1/auth/logout` | Revoke current refresh session |

| POST | `/api/v1/auth/password/forgot` | Start password recovery |

| POST | `/api/v1/auth/password/reset` | Consume reset token and change password |

| GET | `/api/v1/users/me` | Read own profile |

| PATCH | `/api/v1/users/me` | Update own permitted fields |

| PATCH | `/api/v1/users/:userId/status` | Admin account-status operation |

## 4. Important integration dependencies

The Task 3 database design document identifies User, UserRole, RefreshToken, PasswordResetToken and CustomerProfile, but does not fully prescribe every field name. This starter expects:

• `User.email` unique, `displayName`, `phone`, `passwordHash`, `status`, `createdAt`, `updatedAt`.

• `User.userRoles` relation with `role`.

• `RefreshToken` with `userId`, `tokenHash`, `expiresAt`, `revokedAt`.

• `PasswordResetToken` with `userId`, `tokenHash`, `expiresAt`, `usedAt`.

Reconcile these against the actual Task 3 Prisma schema before generating Prisma Client. prisma/task2-required-models.prisma is an integration reference, not a complete schema.

## 5. Items requiring project-specific completion

1. Add the approved email/SMS provider to deliver password-reset links. The starter intentionally does not return or log reset tokens.

2. Confirm whether new accounts are active immediately or require email/phone verification.

3. Confirm the role and account-status enum names.

4. Add AuditLog writes for administrator status changes.

5. Review refresh-cookie CSRF strategy and production CORS origins.

6. Add login abuse monitoring, lockout policy and structured redacted logging.

7. Add integration tests using a disposable PostgreSQL database.

8. Confirm account deletion, data retention and KYC rules.

## 6. Verification status

The files have been generated and packaged. They have not been installed, compiled, connected to PostgreSQL, migrated or run against the actual project schema in this environment. The included Vitest tests cover request validation schemas only. Do not report the authentication service as runtime-tested until the commands in README.md and database integration tests pass in the project repository.

## 7. Recommended next action

Place the source folder into the backend repository, reconcile the Prisma model/field names, install dependencies, generate the Prisma client, then run type checking and tests. Resolve any generated-client/type errors before connecting the frontend.